

Password Storage: The "Do Not Hack Me" Checklist

Understanding Hashing vs. Encryption

Hashing (One-Way)

- **Purpose:** Verify data hasn't changed
- **Reversible:** NO (by design)
- **Use for:** Passwords, file integrity, digital signatures

Encryption (Two-Way)

- **Purpose:** Hide data temporarily
- **Reversible:** YES (with key)
- **Use for:** Credit cards, messages, sensitive data storage

Rule: Never encrypt passwords. Always hash them.

Deprecated Algorithms (Do NOT Use for Passwords)

MD5 (Message Digest 5)

Status:  Broken since 2004

Problems:

- Too fast (attacker can try billions of passwords/second)
- Collision attacks possible
- No built-in salt support

Speed on Modern GPU:

- 200+ billion MD5 hashes/second
- Entire 8-character password space: ~2 hours

Still OK for:

- File checksums (integrity, not security)
- Non-security cache keys

Never use for:

- Passwords
 - Digital signatures
 - Security-critical applications
-

SHA-1 (Secure Hash Algorithm 1)

Status: 🚨 Mathematically broken (2017)

Problems:

- Collision attacks demonstrated
- Too fast for password hashing
- Deprecated by NIST

Example Attack: Google created two different PDFs with identical SHA-1 hash (SHattered attack)

Migration deadline: Most systems must stop using SHA-1 by 2030

Plain SHA-256 (without salt)

Status: ⚠️ Not broken, but insufficient for passwords

Problems:

- Too fast (same GPU can do 10+ billion/second)
- Rainbow table attacks without salt
- No memory hardness

Example Rainbow Table Attack:

None

User password: "password123"

SHA-256: ef92b778bafe771e89245b89ecbc0847...

Attacker's precomputed table:

password123 → ef92b778bafe771e89245b89ecbc0847...

✅ Match found in milliseconds

OK for:

- File integrity
 - Blockchain proof-of-work
 - Non-password data verification
-

✓ Recommended Standards for Password Storage

1. Argon2id (Best Overall - 2023 Winner)

Status: 🏆 Winner of Password Hashing Competition

Why It's Best:

- Memory-hard (resists GPU/ASIC attacks)
- Configurable time cost
- Side-channel resistant
- Hybrid mode (combines Argon2i + Argon2d)

Parameters:

None
Memory: 64 MB minimum (65536 KB)
Iterations: 3+ (depends on acceptable delay)
Parallelism: 4 (CPU threads)

Use Case: New projects, high-security applications

Implementation:

```
JavaScript
// Node.js example
const argon2 = require('argon2');

// Hashing
const hash = await argon2.hash('userPassword', {
  type: argon2.argon2id,
  memoryCost: 65536, // 64 MB
  timeCost: 3,
  parallelism: 4
});

// Verification
const isValid = await argon2.verify(hash, 'userPassword');
```

2. bcrypt (Industry Standard)

Status:  Battle-tested for 25+ years

Why It's Good:

- Adaptive (cost factor increases over time)
- Slow by design (prevents brute force)
- Built-in salt
- Widely supported

Cost Factor (rounds):

None
Cost 10 = ~0.1 seconds (current minimum)
Cost 12 = ~0.4 seconds (recommended 2025)
Cost 14 = ~1.6 seconds (high security)

Every +1 doubles the time

Limitations:

- Maximum password length: 72 bytes
- Not memory-hard (can be attacked with ASICs)

Use Case: Most web applications, existing systems

Implementation:

```
JavaScript
// Node.js example
const bcrypt = require('bcrypt');
const saltRounds = 12;

// Hashing
const hash = await bcrypt.hash('userPassword', saltRounds);
// → $2b$12$EXRkfkdmXSGvH2zj9rqXE0...

// Verification
const isValid = await bcrypt.compare('userPassword', hash);
```

```
Python
# Python example
import bcrypt

# Hashing
password = b"userPassword"
hash = bcrypt.hashpw(password, bcrypt.gensalt(rounds=12))

# Verification
is_valid = bcrypt.checkpw(password, hash)
```

3. PBKDF2 (Acceptable - NIST Approved)

Status:  Acceptable, but weaker than Argon2/bcrypt

Why It's OK:

- NIST-approved
- FIPS 140-2 compliant
- Widely available

Why It's Not Best:

- Not memory-hard
- More vulnerable to GPU attacks than bcrypt
- Requires careful configuration

Minimum Parameters (2025):

```
None
Algorithm: PBKDF2-HMAC-SHA256
Iterations: 600,000+ (OWASP recommendation)
Salt: 128 bits (16 bytes)
Output: 256 bits (32 bytes)
```

Use Case: Government systems, FIPS compliance required

Implementation:

```
JavaScript
// Node.js example
```

```
const crypto = require('crypto');

function hashPassword(password, salt) {
  return crypto.pbkdf2Sync(
    password,
    salt,
    600000, // iterations
    32,     // key length
    'sha256'
  );
}

// Usage
const salt = crypto.randomBytes(16);
const hash = hashPassword('userPassword', salt);
```

The Critical Rule: Always Salt

What is a Salt?

A random value added to each password before hashing.

Without Salt:

```
None
User A: "password123" → hash1
User B: "password123" → hash1 (same!)

Attacker cracks once, gets all passwords
```

With Salt:

```
None
User A: "password123" + "randomSalt1" → hashA
User B: "password123" + "randomSalt2" → hashB (different!)

Attacker must crack each password individually
```

Salt Requirements

- **Unique:** Every password gets its own salt
- **Random:** Use cryptographically secure random generator
- **Length:** Minimum 16 bytes (128 bits)
- **Storage:** Store salt in plain text alongside hash

Example Storage:

```
None
user_id | password_hash | salt
1       | $2b$12$...    | (built-in to bcrypt hash)
2       | $argon2id$... | (built-in to Argon2 hash)
```

Implementation Comparison

Quick Reference Table

Algorithm	Speed	Memory Hard	GPU Resistant	Recommendation
MD5	⚡⚡⚡⚡⚡	✗	✗	🚫 Never use
SHA-1	⚡⚡⚡⚡	✗	✗	🚫 Never use
SHA-256 (plain)	⚡⚡⚡⚡	✗	✗	⚠️ Not for passwords
PBKDF2	⚡⚡	✗	⚠️	✅ Acceptable
bcrypt	⚡	⚠️	✅	✅ Good
Argon2id	⚡	✅	✅	🏆 Best

Common Mistakes

✗ Mistake 1: Hashing Client-Side Only

```
JavaScript
// ✗ WRONG: Hash on client, send to server
```

```
const hash = sha256(password);
fetch('/login', { body: { password: hash } });

// Problem: Hash becomes the password
// If DB leaks, attacker can login with hash directly
```

✅ **Correct:** Hash on server, always transmit over HTTPS

❌ Mistake 2: Global Salt

```
JavaScript
// ❌ WRONG: Same salt for all users
const SALT = 'myAppSalt123';
const hash = bcrypt.hash(password + SALT);

// Problem: One rainbow table cracks all passwords
```

✅ **Correct:** Unique salt per password (bcrypt/Argon2 do this automatically)

❌ Mistake 3: Weak Cost Factor

```
JavaScript
// ❌ WRONG: Cost too low (2010 recommendation)
const hash = bcrypt.hash(password, 10);

// Problem: Modern GPUs crack this easily
```

✅ **Correct:** Cost 12+ for bcrypt (2025 standard)

❌ Mistake 4: Storing Salt Separately (and securely)

```
JavaScript
// ❌ WRONG (misunderstanding)
"I'll encrypt the salt to make it more secure"
```



```
// Problem: Salt is meant to be public
// Its purpose is uniqueness, not secrecy
```

✓ **Correct:** Store salt in plain text. Security comes from the algorithm, not salt secrecy.

✗ Mistake 5: Truncating Passwords

```
JavaScript
// ✗ WRONG
const truncated = password.substring(0, 20);
const hash = bcrypt.hash(truncated);

// Problem: Reduces password space
// "MySecurePassword12345678901234567890" becomes "MySecurePassword1234"
```

✓ **Correct:** Hash the full password (respect algorithm limits, e.g., bcrypt 72 bytes)

Migration Strategy

Moving from Weak to Strong Hashing

Option 1: Forced Reset (Most Secure)

```
None
1. Mark all password hashes as "legacy"
2. Force all users to reset passwords
3. Hash new passwords with Argon2id
4. Delete legacy hashes after reset
```

Option 2: Gradual Migration (User-Friendly)

```
None
1. Keep existing hashes
2. On successful login:
  a. Verify against old hash (MD5/SHA-256)
  b. Re-hash password with Argon2id
```

- c. Update database
- d. Mark user as "migrated"
- 3. After 6 months, force reset for non-migrated users

Implementation:

```
JavaScript
async function verifyAndMigrate(username, password) {
  const user = await db.getUser(username);

  if (user.hashType === 'legacy_md5') {
    // Verify with old method
    const oldHash = md5(password);
    if (oldHash === user.passwordHash) {
      // Upgrade to Argon2id
      const newHash = await argon2.hash(password);
      await db.updateUser(username, {
        passwordHash: newHash,
        hashType: 'argon2id'
      });
      return true;
    }
  } else if (user.hashType === 'argon2id') {
    // Verify with new method
    return await argon2.verify(user.passwordHash, password);
  }

  return false;
}
```

Additional Security Measures

1. Rate Limiting

Prevent brute-force attacks:

```
None
Max 5 attempts per IP per 15 minutes
Max 10 attempts per username per hour
Exponential backoff after failures
```

2. Account Lockout

None

After 10 failed attempts:

- Lock account for 30 minutes
- Send email notification
- Require CAPTCHA on unlock

3. Breach Detection

None

- Check against HaveIBeenPwned API
- Reject commonly leaked passwords
- Force reset if password found in breach

4. Multi-Factor Authentication (MFA)

None

Even with strong hashing:

- SMS/Email codes
- TOTP (Google Authenticator)
- Hardware keys (YubiKey)
- Biometric (FaceID, fingerprint)

Password Policy Recommendations

Minimum Requirements (2025)

- **Length:** 12+ characters
- **Complexity:** No specific requirements (length matters more)
- **Expiry:** Only for privileged accounts (90 days)
- **Reuse:** Prevent last 5 passwords

Do NOT Require

- ❌ Mandatory special characters (reduces password space)
- ❌ Mandatory numbers (users just add "1" at end)
- ❌ Mandatory upper/lowercase (predictable patterns)
- ❌ Regular rotation for normal users (causes weak passwords)

DO Require

- ☒ Minimum 12 characters
 - ☒ Check against breach databases
 - ☒ No common passwords ("password123", "qwerty")
 - ☒ No personal info (name, birthday)
-

Testing Your Implementation

Checklist

- ☐ Using Argon2id, bcrypt, or PBKDF2 (600k+ iterations)?
- ☐ Unique salt per password?
- ☐ Cost factor appropriate for 2025?
- ☐ Passwords transmitted over HTTPS only?
- ☐ Rate limiting implemented?
- ☐ MFA available?
- ☐ Password breach check integrated?
- ☐ Migration plan from legacy hashes?

Penetration Testing

Shell

```
# Test hash strength with hashcat
hashcat -m 3200 -a 3 hashes.txt ?a?a?a?a?a?a?a

# If cracks quickly → algorithm too weak
# If takes days → good
```

Regulatory Compliance

GDPR (Europe)

- "Appropriate security measures" required
- Weak hashing = potential violation
- Data breach notification required

CCPA (California)

- "Reasonable security" standard
- Using MD5/SHA-1 is NOT reasonable

PCI DSS (Credit Card)

- Requirement 8.2.1: Strong cryptography
- bcrypt, Argon2, or PBKDF2 required

HIPAA (Healthcare)

- "Addressable" encryption requirement
 - Strong hashing considered best practice
-

Need a Security Audit?

Worried about security vulnerabilities?

Devxhub provides comprehensive security assessments:

- ✓ Penetration testing
- ✓ Code security review
- ✓ Vulnerability scanning
- ✓ Compliance audits (PCI DSS, HIPAA, GDPR)
- ✓ Security training for teams

Our Security Services:

- Password policy implementation
- Authentication system design
- OAuth 2.0 / OpenID Connect setup
- Zero-trust architecture
- Incident response planning

✉ Contact us: info@devxhub.com

🌐 Visit: www.devxhub.com

☎ Call: +880 1326 506464

Quick Reference

For New Projects:

None

Use: Argon2id

Memory: 64 MB

Iterations: 3

Parallelism: 4

For Existing Projects:

None

Use: bcrypt

Cost: 12+

Migrate to Argon2id when possible

Never Use:

None

MD5, SHA-1, plain SHA-256